

基于区块链的众筹系统

实验要求

1. 功能要求

- **项目创建**：允许用户提交项目名称、描述、筹款目标金额和截止日期 等信息。合约需要存储这些项目信息，并为每个项目分配一个唯一的 ID。
- **资金捐赠**：允许用户选择一个正在进行中的项目，并向其捐赠一定数量的币，存入合约中。合约需要记录每个捐赠者的地址和捐赠金额，并更新项目的当前筹款总额。能够展示所有正在进行和已结束的众筹项目的信息，包括名称、描述、目标金额、已筹金额、剩余时间和捐赠者列表 (可选)。
- **筹款结束逻辑**：基于时间结束，当达到项目的截止日期时，任何人都可以调用一个合约函数来结束该项目。结束结果判断：
 - 达到或超过目标：合约将当前筹集到的所有资金转移给项目发起人。
 - 未达到目标：合约允许捐赠者提取他们捐赠的资金。
- **提取资金**：项目发起人在项目成功结束后，可以调用一个合约函数将筹集到的资金提取到他们的账户。捐赠者在项目失败后，可以调用一个合约函数将他们捐赠的资金提取回来。
- **阶段性资金释放**：基于目标完成度的里程碑式资金释放，捐款达到一定比例后/在项目发起人标记完成某个里程碑后，允许释放一部分资金。
- **早期捐赠奖励机制**：基于时间鼓励捐款人尽早捐赠，可以获得额外的感谢或未来可能的福利。

2. 可选形式

- 控制台调用展示
- 编译器调用展示
- 前端展示

实验过程

本次实验使用github进行版本控制，主要经历了以下阶段：

- a. 搭建 `hardhat` 框架，编写 `CrowdFund` 智能合约
- b. 实现简单前端 `index.html`
- c. 将前端迁移到 `react`，实现控制台和不同用户的权限展示
- d. 尝试加入里程碑资金释放和早期捐赠奖励机制

智能合约设计与实现

- 总体设计和核心模块
 - **基础众筹功能**：包括项目创建、资金募集、项目成功/失败判定及相应的退款逻辑。
 - **投票驱动的里程碑资金释放机制**：项目资金不再一次性全部给予项目方，而是分阶段释放。每次释放都必须经过捐赠人按其捐款额度加权投票同意，有效防止项目方滥用资金。
 - **早期支持者激励机制**：自动记录并识别项目的前 N 名捐赠者，为社区激励和未来空投等活动提供链上凭证，以鼓励早期参与。
- 关键数据结构
 - **Project 结构体**：这是描述一个众筹项目的核心。它不仅包含了如 name (项目名称)、goalAmount (目标金额)、deadline (截止日期) 等基本信息，还通过 state (一个 enum 类型) 来精确追踪项目的生命周期 (如 Fundraising, Successful, Failed)。此外，它还包含一个 Milestone 数组，用于关联其所有的资金释放里程碑。
 - **Milestone 结构体**：这是实现社区治理的关键。每个 Milestone 实例代表一次资金释放请求，它存储了 description (里程碑描述)、releaseAmount (申请释放金额)，以及用于投票的 yesVotes 和 noVotes。其中，voters 这个 mapping 结构至关重要，它记录了每个捐赠者是否已对该里程碑投过票，以防止重复投票。
 - **MilestoneInfo 结构体**：由于 Solidity 的语言限制，包含 mapping 的结构体不能作为函数的返回值。因此，我们创建了一个不含 mapping 的镜像结构体 MilestoneInfo，专门用于 **view** 函数，以便将里程碑的数据安全、高效地暴露给前端 DApp。

- **核心 Mappings:**
 - projects: mapping(uint => Project), 作为合约的“主数据库”, 通过唯一的项目ID索引所有项目数据。
 - contributions: mapping(uint => mapping(address => uint)), 一个二维映射, 精确记录了每个地址对每个项目的捐款总额。这是实现按捐款额加权投票的基础。
 - earlyBirds: mapping(uint => address[]), 为每个项目存储一个地址数组, 用于永久记录早期支持者的身份。
- 核心功能与函数逻辑
 - **模块一：项目生命周期管理**
 - createProject(): 此函数负责项目的初始化。值得注意的是, 除了接收基本信息, 它还接收里程碑描述和金额的数组。在函数内部, 我们对里程碑的总释放金额是否超过目标金额进行了校验, 确保了项目经济模型的合理性。此外, 我们采用了 storage 指针分步赋值的方式来创建 Milestone 实例, 以规避 Solidity 对包含 mapping 的结构体的构造限制。
 - checkDeadlineAndFinalize(): 这是一个任何人都可以调用的公共函数, 用于在项目截止后更新其状态。它通过比较 raisedAmount 和 goalAmount, 将项目状态公平地推进到 Successful 或 Failed, 是触发后续投票或退款流程的“裁判员”。
 - **模块二：社区治理与投票机制**
 - contribute(): 此函数不仅处理捐款逻辑, 还集成了两大关键机制。其一, 它通过检查 contributions mapping 来识别是否为新捐赠者, 并据此更新 earlyBirds 列表。其二, 它累加的捐款额 (contributions) 直接作为该捐赠者在未来投票中的权重, 这是我们防御**女巫攻击** (Sybil Attack) 的核心策略。
 - startMilestoneVote() -> voteOnMilestone() -> executeMilestone(): 这三个函数构成了一个完整的投票工作流。首先, startMilestoneVote 只能由项目创建者调用, 用于开启一个投票窗口。其次, voteOnMilestone 允许任何有贡献的捐赠者投票, 并通过累加其捐款额到 yesVotes 或 noVotes 来实现加权投票。最后, 投票结束后, 任何人都可以调用 executeMilestone, 该函数会根据 yesVotes > project.raisedAmount / 2 这一公开透明的规则来判定投票结果并自动执行资金转移或标记投票失败。
 - **模块三：安全与资金保障**
 - getRefund(): 在项目失败的情况下, 此函数保障了捐赠者的权益, 允许他们安全地取回自己的全部捐款。
 - **重入攻击防范**: 在 executeMilestone 函数中, 我们遵循了‘检查-生效-交互’ (Checks-Effects-Interactions) 的安全模式。即先更新状态 (milestone.executed = true), 然后再执行外部调用 (project.creator.call {...})。这种模式有效防止了**重入攻击**, 确保了合约的资金安全。
- 未来维护方向
 - 进一步扩展合约, 引入更复杂的二次方投票机制以追求更高阶的公平性, 或集成治理代币来实现对平台规则本身的去中心化管理。

前端架构与交互逻辑设计

- 前端技术选型

为了给用户提供一个现代、响应迅速且功能强大的交互界面, 本项目前端部分经过深思熟虑, 选用了以 **React** 为核心的现代 Web 技术栈。选择 React 的主要原因在于其强大的**组件化**能力和基于 **Hooks** 的函数式状态管理模型。这使得我们能够将复杂的用户界面拆解为独立的、可复用的逻辑单元, 极大地提升了开发效率和代码的可维护性。

本应用的前端架构严格遵循**关注点分离 (Separation of Concerns)** 的设计哲学。我们将应用的不同职责清晰地划分到独立的模块中, 避免了代码的耦合与混乱。这种模块化的设计思想体现在我们的目录结构中, 每个目录都承载着单一且明确的职责, 共同构成了一个健壮、可扩展的前端应用。

- 核心目录与模块解析
 - **componmnts**: 此目录是构成用户界面的基础。我们将所有可复用的 UI 元素抽象为独立的 React 组件。
 - **ProjectCard.js**: 这是本应用中最为核心和复杂的“智能组件”。它不仅仅是一个静态的展示卡片, 更是一个**独立的业务逻辑单元**。它负责:
 1. **数据展示**: 接收一个 project 对象, 并将其所有属性 (如名称、目标、进度条) 格式化并渲染。
 2. **状态管理**: 使用 useState 和 useEffect 独立管理其内部状态, 如里程碑数据 (milestones) 和操作加载状态 (loadingAction)。
 3. **业务逻辑**: 内部包含了复杂的条件渲染逻辑, 用于根据项目状态 (project.state)、用户身份 (isCreator) 和投票状态, 动态地显示“发起投票”、“同意/反对”等不同的交互按钮。

4. **合约交互**: 通过引入 useContract hook, 它能直接发起与里程碑相关的合约调用, 实现了业务逻辑的高度内聚。

- **ProjectList.js**: 一个“展示型组件”, 其唯一职责是接收一个项目数组 (projects), 并循环渲染出多个 ProjectCard 组件, 同时负责处理列表为空或加载中的占位显示。

- **CreateProject.js & ContributeModal.js**: 这两个是典型的表单组件, 负责处理用户输入、客户端校验, 并在用户提交时调用上层传递的事件处理器。

- **hooks**: 这是本应用实现状态全局管理和逻辑复用的“心脏”。通过自定义 React Hooks, 我们将复杂的、有副作用的逻辑从 UI 组件中抽离出来。

- **useWeb3.js: Web3 连接管理器**。它封装了所有与 window.ethereum (即 MetaMask 等钱包提供者) 的直接交互。

- **职责**: 自动检查钱包连接状态、处理用户的连接请求 (connect())、获取当前账户地址和 signer 对象, 并对钱包的 accountsChanged 和 chainChanged 事件进行监听, 以实现账户切换或网络切换时的自动响应。

- **价值**: 将DApp与钱包的底层连接逻辑完全隔离, 任何组件只需调用 const { account, connect } = useWeb3() 即可轻松获取所需信息和功能, 无需关心其实现细节。

- **useContract.js: 链上数据与事件的同步中心**。这是连接前端与智能合约数据层的关键桥梁。

- **职责**:

1. **合约实例化**: 利用 useWeb3 提供的 provider 和 signer, 初始化一个可读写的 Ethers.js 合约实例。

2. **数据聚合获取**: 它封装了 getAllProjects, getUserCreatedProjects, getUserContributedProjects 等多个异步数据获取函数, 并通过 Promise.all 并发执行, 高效地拉取所有链上数据。

3. **事件驱动的实时更新**: 这是其最核心的价值。它通过 useEffect 注册了对合约所有关键事件 (如 ContributionMade, MilestoneVoteStarted 等) 的监听。一旦链上发生任何相关事件, 监听器会捕捉到信号并自动调用 refreshData() 函数, 从而重新获取链上最新数据, 并驱动整个 React 应用的UI自动更新。这为用户提供了无需手动刷新的、实时的交互体验。

- **utils**: 此目录提供了纯粹的、无副作用的工具函数, 是前端与区块链交互的“翻译层”和“工具箱”。

- **contractUtils.js: 合约函数调用封装层**。此文件为智能合约中的每一个 public 或 external 函数都提供了一个对应的 JavaScript async 函数。

- **职责**: 它负责处理参数的格式转换 (例如, 将用户输入的 ETH 字符串转换为 Wei 格式的 BigInt)、构建交易对象、发送交易 (await contract.someFunction(...))、等待交易确认 (await tx.wait()), 并进行统一的错误捕获和日志记录。

- **价值**: 这种封装使得业务逻辑层 (如 ProjectCard.js) 的调用变得极其简单和干净, 只需 await startMilestoneVote(contract, ...), 而无需关心底层的 Ethers.js 实现细节。

- **web3Utils.js: 通用数据处理工具**。负责处理与区块链数据格式相关的、与具体合约无关的通用转换, 如将 BigInt 格式的 Wei 转换为用户可读的 ETH 字符串 (formatEther), 或将时间戳转换为本地化日期字符串 (getRemainingTime)。

- 全栈交互核心工作流: 一次完整的投票

为了具体展示本 DApp 的全栈交互模式, 我们以一次完整的“里程碑投票”流程为例, 该流程清晰地体现了从用户界面到智能合约再返回到用户界面的数据闭环。

假设: 项目已成功, 项目创建者 (账户A) 已登录, 捐款人 (账户B) 已捐款。

1. **渲染投票发起界面 (前端)**:

- ProjectCard 组件渲染时, useEffect 触发, 调用 useContract hook 中的 getProjectMilestones() 函数。
- 该函数通过 utils/contractUtils.js 与链上合约交互, 获取到里程碑数据。
- 数据返回后, ProjectCard 重新渲染。在渲染 renderMilestones() 部分时, 内部逻辑判断 isCreator 为 true 且该里程碑未执行, 因此为账户A显示发起投票按钮。

2. **发起投票交易 (前端 -> 钱包 -> 合约)**:

- 账户A点击“发起投票”按钮, 触发 handleStartVoteClick()。
- window.prompt() 弹出, 获取用户输入的投票时长 (例如7天)。
- 该时长被转换为秒后, 作为参数传递给 utils/contractUtils.js 中的 startMilestoneVote() 函数。
- startMilestoneVote() 使用 Ethers.js 构建交易, 并通过 signer 发送给 MetaMask。
- MetaMask 请求用户签名确认。用户确认后, 交易被发送到区块链。

3. **链上状态变更与事件广播 (合约)**:

- 智能合约的 startMilestoneVote 函数被执行。它首先验证调用者身份和项目状态，然后更新对应里程碑的 voteDeadline，最后通过 emit MilestoneVoteStarted(...) 向全网广播一个事件。

4. 实时UI自动更新 (合约 -> 前端):

- useContract.js hook 中监听 MilestoneVoteStarted 事件的监听器被激活。
- 它立即调用 refreshData()，该函数重新从链上获取所有项目和里程碑的最新状态。
- ProjectCard 组件由于获取到的 milestones 数据发生了变化（voteDeadline不再是0），从而自动重新渲染。
- 现在，界面上该里程碑的状态会显示为“投票进行中...”，并出现截止时间。

5. 捐款人投票 (前端 -> ... -> 前端):

- 捐款人账户B登录，看到“投票进行中...”的状态。
- ProjectCard 的渲染逻辑判断 hasContributed 为 true 且 !isCreator，因此为账户B显示 👍 同意 / 👎 反对按钮。
- 账户B点击“同意”，重复步骤 2-4，但这次调用的是 voteOnMilestone() 函数。
- useContract hook 捕捉到 VotedOnMilestone 事件，再次自动刷新UI，用户能立刻看到“同意票数”的权重增加了。

实验环境与工具链

为了确保开发、测试和部署流程的高效与可靠，本项目搭建了一套现代化的 DApp 开发工作流，其核心工具链包括：

- **Node.js:** 作为 JavaScript 的运行时环境，是整个开发环境的基础。
- **Hardhat:** 一个强大的以太坊开发环境。在本项目中，它主要承担了以下职责：
 - **合约编译:** 将 .sol 源码编译成 EVM 字节码和 ABI。
 - **本地网络:** 通过 npx hardhat node 启动一个功能完备的本地以太坊节点，用于快速、无成本的开发和调试。
 - **部署脚本:** 利用 scripts/deploy.js 自动化合约部署流程。
- **React (Create React App):** 用于构建用户界面的前端框架，提供了组件化开发、状态管理和热重载等核心功能。
- **Ethers.js:** 一个强大而完整的以太坊钱包实现和与区块链交互的 JavaScript 库。它是连接我们 React 应用和智能合约的关键桥梁。
- **MetaMask:** 作为浏览器钱包插件，是用户与 DApp 交互的入口。我们通过将其连接到本地 Hardhat 网络，实现了对不同账户（项目方、捐款人）的模拟和测试。
- **自动化脚本 (Shell Scripts):** 为了简化重复性的环境启动和部署工作，我们编写了 start_dev_environment.sh 脚本，它能一键完成清理、编译、启动节点、部署合约和更新前端配置等一系列操作，极大地提升了开发效率。

功能测试与结果展示

为了全面验证本 DApp 的功能完整性和逻辑正确性，我们设计了以下端到端的测试用例。这些用例将通过在前端界面上扮演不同角色（项目创建者、捐款人）来执行。

• 测试用例 1：完整的项目生命周期

1. **[创建项目]** - 使用**账户A**登录，访问‘我的项目’页面，填写项目信息及至少两个里程碑，并成功创建项目。
2. **[捐款与早期奖励]** - 依次切换到**账户B**、**账户C**、**账户D**，分别对该项目进行捐款。验证‘早期支持者’区域是否正确显示前三位捐赠者的地址。
3. **[项目结算]** - 将一个项目的截止日期设置为过去时间，或等待其自然过期。使用任意账户访问该项目，验证‘结算项目’按钮是否出现。点击该按钮，确认交易后，验证项目状态是否从‘筹款中’变为‘成功’。

• 测试用例 2：里程碑投票与资金释放

1. **[发起投票]** - 确保项目状态为‘成功’。使用**项目创建者（账户A）**登录，在第一个里程碑上点击‘发起投票’，输入自定义时长并发起投票。验证该里程碑状态是否变为‘投票进行中’。
2. **[捐款人投票]** - 分别使用**捐款人（账户B、C）**登录，对正在投票的里程碑进行投票（例如，都投同意票）。验证每次投票后，‘同意票数’的权重是否正确增加，并且已投票的用户不再看到投票按钮。
3. **[执行投票]** - 在投票截止后，使用任意账户点击‘执行投票结果’。验证投票是否被判定为成功，资金是否被正确转移给项目创建者（可在 MetaMask 中检查账户A余额变化），以及该里程碑状态是否更新为‘已执行’。

- **测试用例 3：失败与退款流程**

1. 创建一个新项目，但只让少数用户捐款，确保其在截止时无法达到筹款目标。
2. 结算该项目，验证其状态是否变为‘失败’。
3. 使用**捐款人账户**登录，验证‘申请退款’按钮是否出现。点击该按钮，验证资金是否成功退回到捐款人钱包。

结论与心得体会

- **实验结论:**

- 本次实验成功地设计并实现了一个功能完善的、基于区块链的去中心化众筹平台。通过结合 React 前端框架和 Solidity 智能合约，我们不仅实现了传统众筹的核心功能，还创新性地引入了由捐赠者投票驱动的里程碑资金释放机制和早期支持者激励系统。端到端的测试结果表明，所有功能模块均按预期工作，系统逻辑严谨，交互流程顺畅。
- 特别地，按捐款额加权的投票机制被证明是防御女巫攻击的有效手段，而事件驱动的前端自动刷新架构，则为用户提供了媲美传统 Web 应用的实时交互体验。

- **心得与反思:**

- 在本次实验中，我深刻体会到 DApp 开发的复杂性与严谨性。与传统开发最大的不同在于，智能合约一旦部署，其逻辑便难以更改，这要求在设计阶段就必须对所有可能性和安全漏洞（如重入攻击）进行周全的考虑。
- 前端与合约的交互也充满了挑战。我遇到了包括 **ABI 不同步**、**库版本不兼容 (Ethers.js v5 vs v6)**、**数据类型不匹配 (BigInt vs Number)** 以及 Solidity 语言限制（如 mapping in struct）*在内的多种问题。通过系统性的调试方法，如添加日志、检查网络请求、利用开发者工具，我学会了如何定位并解决这些在 DApp 开发中特有的问题。
- 最大的收获是理解了‘状态’在全栈应用中的流动方式：从前端用户操作，到后端合约状态变更，再通过事件广播回到前端 UI 更新，形成一个完整的响应式闭环。自定义 React Hooks (useWeb3, useContract) 在解耦和管理这种复杂状态流中起到了至关重要的作用。